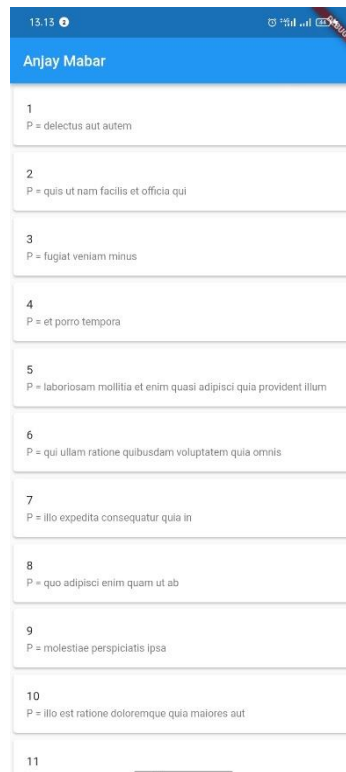


HTTP REQUEST

6.1 Desain Aplikasi

Pada project kali ini anda akan membuat sebuah aplikasi yang mengakses REST API dari JsonPlaceholder. Aplikasi yang dibuat akan menampilkan contoh data API gratis dan menampilkan data sementara untuk tes Aplikasi. Hasil akhir dari aplikasi yang dibuat dapat dilihat pada gambar berikut ini.



Desain yang dibuat mengikuti pola master detail pada praktikum 5 tetapi sekarang berisi data dari REST API.

6.2 Persiapan Project

Untuk membuat project ini anda membutuhkan :

6.2.1 Api Key dari json place holder untuk mendapatkan api key dapat mengikuti tutorial berikut ini :

- <https://jsonplaceholder.typicode.com/>

6.2.2 Library Flutter http

Setelah project flutter berhasil dibuat tambahkan library http ke project tersebut. Untuk menambahkan library bisa menggunakan plugin vscode pubspec assist atau mengikuti tutorial disini

<https://stackoverflow.com/questions/59915827/how-to-add-a-library-to-flutter-dependencies>

6.2.3 Jika ada permintaan update flutter lakukan flutter upgrade.

6.3 Koneksi ke REST API

6.3.1 Menguji koneksi di dashboard api JsonPlaceholder

Untuk menguji koneksi ke REST API themoviedb dapat dilakukan di link berikut ini <https://jsonplaceholder.typicode.com> pada link berikut anda dapat menguji dan membuat request ke server themoviedb dan melihat response nya dalam berbagai format json.

Resources

JSONPlaceholder comes with a set of 6 common resources:

/posts	100 posts
/comments	500 comments
/albums	100 albums
/photos	5000 photos
/todos	200 todos
/users	10 users

Note: resources have relations. For example: posts have many comments, albums have many photos, ... see [guide](#) for the full list.

Routes

All HTTP methods are supported. You can use http or https for your requests.

GET [/posts](#)

6.3.2 Mengkoneksikan Aplikasi Flutter ke themoviedb dengan package http

Untuk mulai mengkoneksikan aplikasi flutter yang dibuat ke rest api json place holder lakukanlah langkah berikut:

6.3.2.1 Untuk mengakses internet kita harus menambahkan permission internet pada android manifest cari lah file android manifest.xml pada folder android/app/src/main/AndroidManifest.xml kemudian tambahkan permission internet.

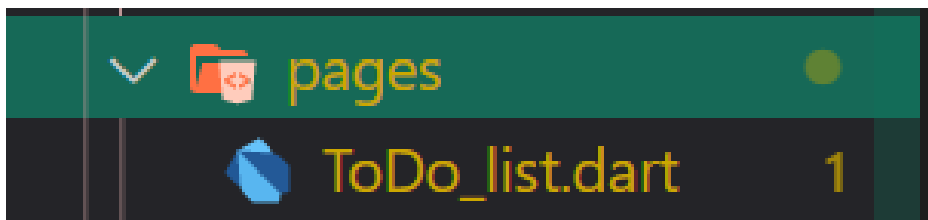
```
11 <manifest xmlns:android="http://schemas.android.com/apk/res/android" package="com.example.moviedb2">
12   <uses-permission android:name="android.permission.INTERNET" />
13   <application android:label="moviedb2" android:icon="@mipmap/ic_launcher">
14     <activity android:name=".MainActivity" android:launchMode="singleTop" android:theme="@style/LaunchTheme" android:
15       <!-- Specifies an Android theme to apply to this Activity as soon as
16           the Android process has started. This theme is visible to the user
17           while the Flutter UI initializes. After that, this theme continues
18           to determine the Window background behind the Flutter UI. -->
19       <meta-data android:name="io.flutter.embedding.android.NormalTheme" android:resource="@style/NormalTheme" />
20       <!-- Displays an Android View that continues showing the launch screen
21           Drawable until Flutter paints its first frame, then this splash
```

6.3.2.2 Hapus komentar yang ada pada file main.dart dari kode program awal.

6.3.2.3 Ubah MyHomePage menjadi stateless widget.

```
1  import 'package:flutter/material.dart';
2
3  Run | Debug
4  void main() {
5    runApp(MyApp());
6  }
7
8  class MyApp extends StatelessWidget {
9    // This widget is the root of your application.
10   @override
11   Widget build(BuildContext context) {
12     return MaterialApp(
13       title: 'Flutter Demo',
14       theme: ThemeData(
15         primarySwatch: Colors.blue,
16       ), // ThemeData
17       home: MyHomePage(),
18     ); // MaterialApp
19   }
20 }
21
22 class MyHomePage extends StatelessWidget {
23   @override
24   Widget build(BuildContext context) {
25     return Container();
26   }
27 }
```

6.3.2.4 Buat file baru baru “pages” pada folder lib dan buat sebuah file dengan nama ToDo_list.dart.



6.3.2.5 Buat sebuah statefull widget pada file ToDo_list.dart

```

1  import 'package:flutter/material.dart';
2
3  class MovieList extends StatefulWidget {
4    @override
5    _MovieListState createState() => _MovieListState();
6  }
7
8  class _MovieListState extends State<MovieList> {
9    @override
10   Widget build(BuildContext context) {
11     return Container();
12   }
13 }
14

```

- 6.3.2.6 Import File movie_list.dart ke main.dart dan gunakan ToDo_list.dart sebagai return dari class MyHomePage

```

20
21 class MyHomePage extends StatelessWidget {
22   @override
23   Widget build(BuildContext context) {
24     return ToDoList();
25   }
26 }

```

- 6.3.2.7 Sebelum melanjutkan ke pembuatan aplikasi kita buat terlebih dahulu sebuah helper class untuk koneksi ke rest api Json Place Holder. Buatlah sebuah folder dengan nama servicedan isi dengan file http_service.dart.

```

import 'package:http/http.dart' as http;

class HttpService {
  final String baseUrl = 'https://jsonplaceholder.typicode.com/todos';
}

```

- 6.3.2.8 Selanjutnya buat sebuah function untuk mengambil response dari server json place holder

```

3
4 import 'dart:io';
5 import 'package:http/http.dart' as http;
6
7
8 class HttpService {
9   final String baseUrl = 'https://jsonplaceholder.typicode.com/todos';
10
11   Future<List> getPopularMovies() async {
12     final String uri = baseUrl;
13
14     http.Response result = await http.get(Uri.parse(uri));
15     if (result.statusCode == HttpStatus.ok) {
16       final jsonResponse = jsonDecode(result.body);
17       List movies = jsonResponse.map((i) => Movie.fromJson(i)).toList();
18       return movies;
19     } else {
20       print("Fail");
21       List.empty();
22       return List.empty();
23     }
24   }
25 }
26

```

6.3.2.9 Update file `ToDo _list.dart` agar dapat menggunakan file `httpService` dan mereturn `kanwidget scaffold`.

6.3.2.10 Pada function `_ToDo ListState` tambahkan variabel berikut ini untuk variabel `servicejangan` lupa import dulu file `httpservice` nya.

```

10
11 class _ToDoListState extends State<ToDoList> {
12   String result = "";
13   late HttpService service;

```

6.3.2.11 Kemudian tambahkan method override `init state` agar permintaan ke rest api dapat dilakukan ketika state di inisialisasi.

```

class _ToDoListState extends State<ToDoList> {
  String result = "";
  late HttpService service;

  @override
  void initState() {
    service = HttpService();
    super.initState();
  }

```

6.3.2.12 Selanjutnya update function `build` pada `ToDoListState` dengan menggunakan `widgetscaffold`.

```

20
21  Widget build(BuildContext context) {
22    service.getToDo().then((value) => {
23      setState(() {
24        result = value;
25      })
26    });
27    return Scaffold(
28      appBar: AppBar(
29        title: Text("Popular Movies"),
30      ), // AppBar
31      body: Container(
32        child: Text(result),
33      ), // Container
34    ); // Scaffold
35  }

```

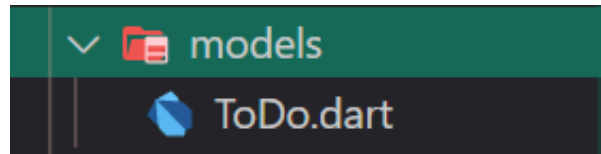
6.3.2.13 Jalankan aplikasi, jika di awal terjadi error reload menggunakan hot restart.



6.3.2.14 Pada langkah ini kita sudah berhasil mendapatkan response dari REST Api ke widget text namun belum tampil dengan baik karena belum menggunakan listview dan models. Selanjutnya kita persiapkan sebuah file models dan listview.

6.3.3 Membuat model untuk response http

6.3.4 Buat folder models didalam folder lib dan isikan dengan file ToDo.dart



6.3.4.1 Di dalam movie.dart buat lah sebuah class movie lengkap dengan variabel dan konstruktor seperti dibawah ini.

```
1 class ToDo {  
2   int? userId;  
3   int? id;  
4   String? title;  
5  
6   ToDo(this.userId, this.id, this.title);  
7 }
```

6.3.4.2 Selanjutnya buatlah sebuah function untuk mengkonversi json menjadi response yang sesuai dengan class ToDo.

```
1 class ToDo {  
2   int? userId;  
3   int? id;  
4   String? title;  
5  
6   ToDo(this.userId, this.id, this.title);  
7  
8   ToDo.fromJson(Map<String, dynamic> parsedJson) {  
9     this.id = parsedJson['id'];  
10    this.title = parsedJson['title'];  
11    this.userId = parsedJson['userId'];  
12  }  
13 }
```

6.3.4.3 Selanjutnya update function http_service.dart menjadi seperti dibawah ini.

```

1  import 'dart:convert';
2  import 'dart:io';
3  import 'package:http/http.dart' as http;
4  import 'package:flutter_application_1/Models/ToDo.dart';
5
6  class HttpService {
7    final String baseUrl = 'https://jsonplaceholder.typicode.com/todos';
8
9    Future<List> getPopularMovies() async {
10     final String uri = baseUrl;
11
12     http.Response result = await http.get(Uri.parse(uri));
13     if (result.statusCode == HttpStatus.ok) {
14       final jsonResponse = jsonDecode(result.body);
15       List movies = jsonResponse.map((i) => Movie.fromJson(i)).toList();
16       return movies;
17     } else {
18       print("Fail");
19       List.empty();
20       return List.empty();
21     }
22   }
23 }
24

```

- 6.3.4.4 Sampai pada tahap ini aplikasi anda tidak dapat berjalan dengan baik, biarkan dan lanjutkan ke langkah selanjutnya untuk membuat list ToDo.

6.4 Membuat halaman list Populer Movie

Untuk membuat list view pada widget movie list anda memerlukan beberapa widget baru antara lain list view dan card. Selain itu juga dibutuhkan data dari http service. Langkah langkah yang perlu dilakukan adalah sebagai berikut :

- 6.4.1 Update inialisasi variabel pada class ToDoListState

```

8  class _ToDoListState extends State<ToDoList> {
9    int movieCount = 0;
10   late List Movies;
11   late HttpService service;

```

- 6.4.2 Tambahkan method initialize() pada class ToDoliststate

```

8  class _ToDoListState extends State<ToDoList> {
9      int movieCount = 0;
10     late List Movies;
11     late HttpService service;
12
13     Future initialize() async {
14         Movies = [];
15         Movies = await service.getPopularMovies();
16         print(Movies);
17         setState(() {
18             movieCount = Movies.length;
19             Movies = Movies;
20         });
21     }

```

6.4.3 Tambahkan function initialize pada initState

```

8  class _ToDoListState extends State<ToDoList> {
9      int movieCount = 0;
10     late List Movies;
11     late HttpService service;
12
13     Future initialize() async {
14         Movies = [];
15         Movies = await service.getPopularMovies();
16         print(Movies);
17         setState(() {
18             movieCount = Movies.length;
19             Movies = Movies;
20         });
21     }
22
23     @override
24     void initState() {
25         service = HttpService();
26         initialize();
27         super.initState();
28     }
29

```

6.4.4 Selanjutnya buat listview dan card berdasarkan data dari initialize

```

29
30 @override
31 Widget build(BuildContext context) {
32   return Scaffold(
33     appBar: AppBar(
34       title: Text("Anjay Mabar"),
35     ), // AppBar
36     body: ListView.builder(
37       itemCount: (this.ToDoCount == null) ? 0 : this.ToDoCount,
38       itemBuilder: (context, int position) {
39         return Card(
40           color: Colors.white,
41           elevation: 2.0,
42           child: ListTile(
43             title: Text(ToDo[position].id.toString()),
44             subtitle: Text('Pro = ' + ToDo[position].title),
45           ), // ListTile
46         ); // Card
47       }, // ListView.builder
48     ); // Scaffold
49   }

```

6.4.5 Lakukan hot restart dan aplikasi akan berubah menjadi seperti dibawah ini.



6.4.6 Selamat anda berhasil membuat listview sekarang saatnya mempercantik listview yang dibuat.

6.4.7 Challenge : carilah cara menambahkan gambar dari response api ke listview, tambahkan gambar tersebut ke listview.

6.5 Membuat halaman detail Populer Movie

6.5.1 Untuk membuat perpindahan dari movie list ke movie detail buatlah onTap event di listview pada movie list.

```
39   return Card(  
40     color: Colors.white,  
41     elevation: 2.0,  
42     child: ListTile(  
43       title: Text(movies[position].title),  
44       subtitle: Text(  
45         'Rating = ' + movies[position].voteAverage.toString(),  
46       ), // Text  
47       onTap: () {  
48         MaterialPageRoute route = MaterialPageRoute(  
49           builder: (_) => MovieDetail(movies[position])); // MaterialPageRoute  
50         Navigator.push(context, route);  
51       },  
52     ), // ListTile  
53   ); // Card
```

6.5.2 Pada event on tap Widget MovieDetail belum dibuat, buatlah widget ini pada folder pages/movie_detail.dart.

```
lib > pages > movie_detail.dart > ...  
1  import 'package:flutter/material.dart';  
2  
3  class MovieDetail extends StatelessWidget {  
4    @override  
5    Widget build(BuildContext context) {  
6      return Container();  
7    }  
8  }
```

6.5.3 Lengkapi Movie Detail untuk menerima parameter Movies

```

lib > pages > movie_detail.dart > ...
1  import 'package:flutter/material.dart';
2  import 'package:moviedb2/models/movie.dart';
3
4  class MovieDetail extends StatelessWidget {
5      final Movie movie;
6      final String imgPath = 'https://image.tmdb.org/t/p/w500/';
7
8      MovieDetail(this.movie);
9
10     @override
11     Widget build(BuildContext context) {
12         return Container();
13     }
14 }
15

```

6.5.4 Lengkapi detail widget.

```

10     @override
11     Widget build(BuildContext context) {
12         String path;
13         if (movie.posterPath != null) {
14             path = imgPath + movie.posterPath;
15         } else {
16             path =
17                 'https://images.freeimages.com/images/large-previews/5eb/movie-clapboard-1184339.jpg';
18         }
19         double height = MediaQuery.of(context).size.height;
20         return Scaffold(
21             appBar: AppBar(
22                 title: Text(movie.title),
23             ), // AppBar
24             body: SingleChildScrollView(
25                 child: Center(
26                     child: Column(
27                         children: [
28                             Container(
29                                 padding: EdgeInsets.all(16),
30                                 height: height / 1.5,
31                                 child: Image.network(path)), // Container
32                             Container(
33                                 child: Text(movie.overview),
34                                 padding: EdgeInsets.only(left: 16, right: 16),
35                             ), // Container
36                         ],
37                     ), // Column
38                 ), // Center
39             ), // SingleChildScrollView
40         ); // Scaffold
41     }

```

6.5.5 Challenge Modifikasilah Tampilan agar terlihat lebih menarik